```
!pip install --upgrade scikit-learn pandas numpy joblib
```

```
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Collecting scikit-learn
  Downloading scikit_learn-1.7.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (11 kB)
Requirement already satisfied: pandas in /usr/local/lib/python3.12/dist-packages (2.2.2)
Collecting pandas
  Downloading pandas-2.3.2-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (91 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 91.2/91.2 kB 3.8 MB/s eta 0:00:00
Requirement already satisfied: numpy in /usr/local/lib/python3.12/dist-packages (2.0.2)
Collecting numpy
  Downloading numpy-2.3.2-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (62 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 62.1/62.1 kB 5.5 MB/s eta 0:00:00
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (1.5.1)
Requirement already satisfied: scipy>=1.8.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.1)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas) (2025.2)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas) (1.
Downloading scikit_learn-1.7.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (9.5 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 9.5/9.5 MB 121.0 MB/s eta 0:00:00
Downloading pandas-2.3.2-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (12.0 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.0/12.0 MB 156.6 MB/s eta 0:00:00
Downloading numpy-2.3.2-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 16.6/16.6 MB 145.1 MB/s eta 0:00:00
Installing collected packages: numpy, pandas, scikit-learn
  Attempting uninstall: numpy
    Found existing installation: numpy 2.0.2
    Uninstalling numpy-2.0.2:
      Successfully uninstalled numpy-2.0.2
  Attempting uninstall: pandas
    Found existing installation: pandas 2.2.2
    Uninstalling pandas-2.2.2:
      Successfully uninstalled pandas-2.2.2
  Attempting uninstall: scikit-learn
    Found existing installation: scikit-learn 1.6.1
    Uninstalling scikit-learn-1.6.1:
      Successfully uninstalled scikit-learn-1.6.1
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This behaviour is
google-colab 1.0.0 requires pandas==2.2.2, but you have pandas 2.3.2 which is incompatible.
opencv-python-headless 4.12.0.88 requires numpy<2.3.0,>=2; python_version >= "3.9", but you have numpy 2.3.2 which is incomp
cupy-cuda12x 13.3.0 requires numpy<2.3,>=1.22, but you have numpy 2.3.2 which is incompatible.
opencv-contrib-python 4.12.0.88 requires numpy<2.3.0,>=2; python_version >= "3.9", but you have numpy 2.3.2 which is incompa
cudf-cu12 25.6.0 requires pandas<2.2.4dev0,>=2.0, but you have pandas 2.3.2 which is incompatible.
opencv-python 4.12.0.88 requires numpy<2.3.0,>=2; python_version >= "3.9", but you have numpy 2.3.2 which is incompatible.
tensorflow 2.19.0 requires numpy<2.2.0,>=1.26.0, but you have numpy 2.3.2 which is incompatible.
dask-cudf-cu12 25.6.0 requires pandas<2.2.4dev0,>=2.0, but you have pandas 2.3.2 which is incompatible.
numba 0.60.0 requires numpy<2.1,>=1.22, but you have numpy 2.3.2 which is incompatible.
Successfully installed numpy-2.3.2 pandas-2.3.2 scikit-learn-1.7.1
```

```
import pandas as pd
import numpy as np
import re
import string
import joblib
import warnings
from pathlib import Path
from typing import Tuple, Dict, Any
import io

# Scikit-learn imports
import sklearn
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import LabelEncoder

# For file upload in Colab
from google.colab import files
import matplotlib.pyplot as plt
import seaborn as sns

# Suppress warnings
warnings.filterwarnings('ignore')

print("All libraries imported successfully!")
print(f"Pandas version: {pd.__version__}")
print(f"Scikit-learn version: {sklearn.__version__}")
```

```
All libraries imported successfully!
Pandas version: 2.2.2
Scikit-learn version: 1.6.1
```

```
# Cell 3: Upload Your CSV File
# ============================
print("Please upload your CSV file (cleaned_shuffled_dataset.csv)")
uploaded = files.upload()
# Get the uploaded file name
csv_filename = list(uploaded.keys())[0]
print(f"Uploaded file: {csv_filename}")
```

```
Please upload your CSV file (cleaned_shuffled_dataset.csv)
```

Choose Files No file chosen Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.

```
Saving cleaned_shuffled_dataset.csv to cleaned_shuffled_dataset.csv
```

```
# Cell 4: Define the ToxicityClassifier Class
# ===========================================
class ToxicityClassifier:
    """
    A comprehensive toxicity classification system using TF-IDF and Logistic Regression.
    Optimized for Google Colab usage.
    """

    def __init__(self, random_state: int = 42):
        """Initialize the classifier with default parameters."""
        self.random_state = random_state
        self.model = None
        self.vectorizer = None
        self.label_encoder = None
        self.is_trained = False

        # TF-IDF parameters optimized for toxicity detection
        self.tfidf_params = {
            'max_features': 8000,   # Reduced for Colab memory
            'ngram_range': (1, 2),
            'min_df': 2,
            'max_df': 0.95,
            'stop_words': 'english',
            'lowercase': True,
            'strip_accents': 'ascii'
        }

        # Logistic Regression parameters
        self.lr_params = {
            'max_iter': 1000,
            'random_state': self.random_state,
            'class_weight': 'balanced',
            'multi_class': 'ovr'
        }

    def preprocess_text(self, text: str) -> str:
        """Preprocess text data for better classification performance."""
        if pd.isna(text) or text is None:
            return ""

        # Convert to string and lowercase
        text = str(text).lower()

        # Remove URLs
        text = re.sub(r'http\S+|www\S+|https\S+', '', text, flags=re.MULTILINE)

        # Remove email addresses
        text = re.sub(r'\S+@\S+', '', text)

        # Remove extra whitespace and newlines
        text = re.sub(r'\s+', ' ', text)

        # Remove leading/trailing whitespace
        text = text.strip()

        return text if text else ""

    def load_and_prepare_data(self, csv_file_path: str) -> Tuple[pd.DataFrame, np.ndarray, np.ndarray]:
        """Load CSV data and prepare it for training."""
        print(f"Loading data from: {csv_file_path}")

        try:
            df = pd.read_csv(csv_file_path)
            print(f"✅ Dataset loaded successfully: {df.shape[0]} rows, {df.shape[1]} columns")
```

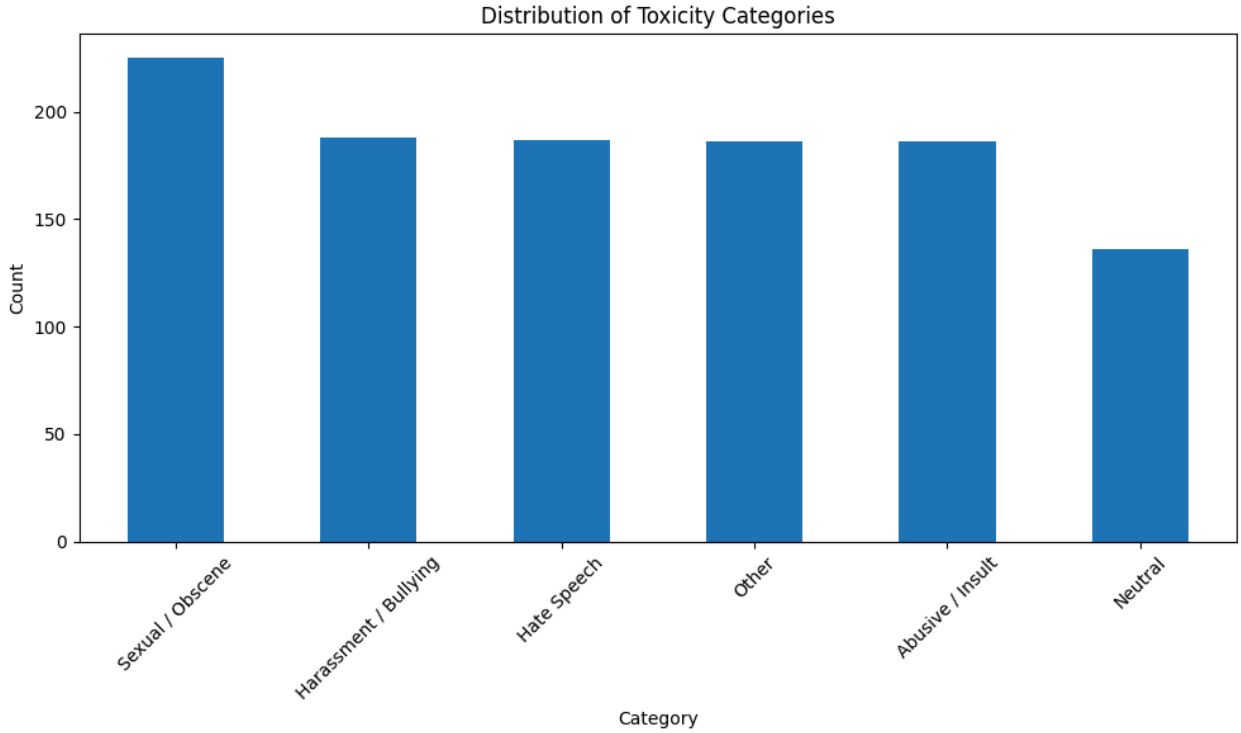
```
        except Exception as e:
            raise Exception(f"Error loading CSV file: {str(e)}")

        # Display basic dataset info
        print(f"\n📊 Dataset Info:")
        print(f"Columns: {list(df.columns)}")
        print(f"\n🏷️ Category distribution:")
        category_counts = df['Category_of_Toxicity'].value_counts()
        print(category_counts)

        # Visualize category distribution
        plt.figure(figsize=(10, 6))
        category_counts.plot(kind='bar')
        plt.title('Distribution of Toxicity Categories')
        plt.xlabel('Category')
        plt.ylabel('Count')
        plt.xticks(rotation=45)
        plt.tight_layout()
        plt.show()

        # Check for missing values
        missing_text = df['Context_or_Example_Sentence'].isna().sum()
        missing_labels = df['Category_of_Toxicity'].isna().sum()

        print(f"\n🔍 Data Quality Check:")
        print(f"Missing text samples: {missing_text}")
        print(f"Missing labels: {missing_labels}")

        # Clean data
        initial_size = len(df)
        df = df.dropna(subset=['Category_of_Toxicity'])
        df['Context_or_Example_Sentence'] = df['Context_or_Example_Sentence'].fillna('')

        print(f"Removed {initial_size - len(df)} rows with missing labels")

        # Preprocess text
        print("\n🔄 Preprocessing text data...")
        df['processed_text'] = df['Context_or_Example_Sentence'].apply(self.preprocess_text)

        # Remove empty texts
        df = df[df['processed_text'].str.len() > 0]
        print(f"✅ Final dataset size: {len(df)} rows")

        return df, df['processed_text'].values, df['Category_of_Toxicity'].values

    def train_model(self, X: np.ndarray, y: np.ndarray, test_size: float = 0.2) -> Dict[str, Any]:
        """Train the toxicity classification model."""
        print("\n" + "="*60)
        print("🚀 TRAINING TOXICITY CLASSIFIER")
        print("="*60)

        # Split data
        X_train, X_test, y_train, y_test = train_test_split(
            X, y, test_size=test_size, random_state=self.random_state, stratify=y
        )

        print(f"📚 Training set size: {len(X_train)}")
        print(f"🧪 Testing set size: {len(X_test)}")

        # Initialize components
        self.vectorizer = TfidfVectorizer(**self.tfidf_params)
        self.label_encoder = LabelEncoder()

        # Encode labels
        y_train_encoded = self.label_encoder.fit_transform(y_train)
        y_test_encoded = self.label_encoder.transform(y_test)

        print(f"\n🏷️ Number of classes: {len(self.label_encoder.classes_)}")
        print(f"Classes: {list(self.label_encoder.classes_)}")

        # Vectorize text
        print("\n🔤 Vectorizing text data...")
        X_train_tfidf = self.vectorizer.fit_transform(X_train)
        X_test_tfidf = self.vectorizer.transform(X_test)

        print(f"📐 TF-IDF matrix shape: {X_train_tfidf.shape}")
        print(f"📖 Vocabulary size: {len(self.vectorizer.vocabulary_)}")

        # Train model
        print("\n🤖 Training Logistic Regression model...")
        self.model = LogisticRegression(**self.lr_params)
        self.model.fit(X_train_tfidf, y_train_encoded)
```

```python

        # Predictions
        y_pred_encoded = self.model.predict(X_test_tfidf)
        y_pred = self.label_encoder.inverse_transform(y_pred_encoded)

        # Calculate metrics
        accuracy = accuracy_score(y_test, y_pred)
        print(f"\n✅ Test Accuracy: {accuracy:.4f}")

        # Classification report
        print("\n📊 Classification Report:")
        print("="*60)
        report = classification_report(y_test, y_pred, output_dict=True)
        print(classification_report(y_test, y_pred))

        # Confusion Matrix Visualization
        plt.figure(figsize=(10, 8))
        cm = confusion_matrix(y_test, y_pred, labels=self.label_encoder.classes_)
        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                    xticklabels=self.label_encoder.classes_,
                    yticklabels=self.label_encoder.classes_)
        plt.title('Confusion Matrix')
        plt.xlabel('Predicted')
        plt.ylabel('Actual')
        plt.xticks(rotation=45)
        plt.yticks(rotation=0)
        plt.tight_layout()
        plt.show()

        # Cross-validation
        print("\n🔄 Cross-Validation Scores:")
        cv_scores = cross_val_score(self.model, X_train_tfidf, y_train_encoded, cv=5)
        print(f"CV Accuracy: {cv_scores.mean():.4f} (+/- {cv_scores.std() * 2:.4f})")

        self.is_trained = True

        return {
            'accuracy': accuracy,
            'classification_report': report,
            'cv_scores': cv_scores,
            'test_predictions': y_pred,
            'test_true': y_test
        }

    def predict_comment(self, text: str) -> Dict[str, Any]:
        """Predict toxicity category for a single comment."""
        if not self.is_trained:
            raise ValueError("Model must be trained before making predictions")

        processed_text = self.preprocess_text(text)

        if not processed_text:
            return {
                'original_text': text,
                'predicted_category': 'Neutral',
                'confidence': 0.0,
                'probabilities': {}
            }

        # Vectorize and predict
        text_tfidf = self.vectorizer.transform([processed_text])
        prediction_encoded = self.model.predict(text_tfidf)[0]
        predicted_category = self.label_encoder.inverse_transform([prediction_encoded])[0]

        # Get probabilities
        probabilities = self.model.predict_proba(text_tfidf)[0]
        prob_dict = {
            self.label_encoder.inverse_transform([i])[0]: float(prob)
            for i, prob in enumerate(probabilities)
        }

        confidence = float(max(probabilities))

        return {
            'original_text': text,
            'processed_text': processed_text,
            'predicted_category': predicted_category,
            'confidence': confidence,
            'probabilities': prob_dict
        }

```

```
✅ ToxicityClassifier class defined successfully!
```

```python
# Cell 5: Load and Explore Your Data
# ==================================
# Initialize classifier
classifier = ToxicityClassifier(random_state=42)

# Load data
try:
    df, X, y = classifier.load_and_prepare_data(csv_filename)
    print(f"\n✅ Data loaded and preprocessed successfully!")

    # Show some sample data
    print(f"\n📋 Sample data:")
    sample_df = df[['Context_or_Example_Sentence', 'Category_of_Toxicity', 'processed_text']].head()
    for idx, row in sample_df.iterrows():
        print(f"\nOriginal: {row['Context_or_Example_Sentence']}")
        print(f"Processed: {row['processed_text']}")
        print(f"Category: {row['Category_of_Toxicity']}")
        print("-" * 40)

except Exception as e:
    print(f"❌ Error: {e}")
```

```
Loading data from: cleaned_shuffled_dataset.csv
✅ Dataset loaded successfully: 1108 rows, 4 columns

📊 Dataset Info:
Columns: ['Language_Type', 'Toxic_Word_or_Phrase', 'Context_or_Example_Sentence', 'Category_of_Toxicity']

🏷️ Category distribution:
Category_of_Toxicity
Sexual / Obscene         225
Harassment / Bullying    188
Hate Speech              187
Other                    186
Abusive / Insult         186
Neutral                  136
Name: count, dtype: int64
```

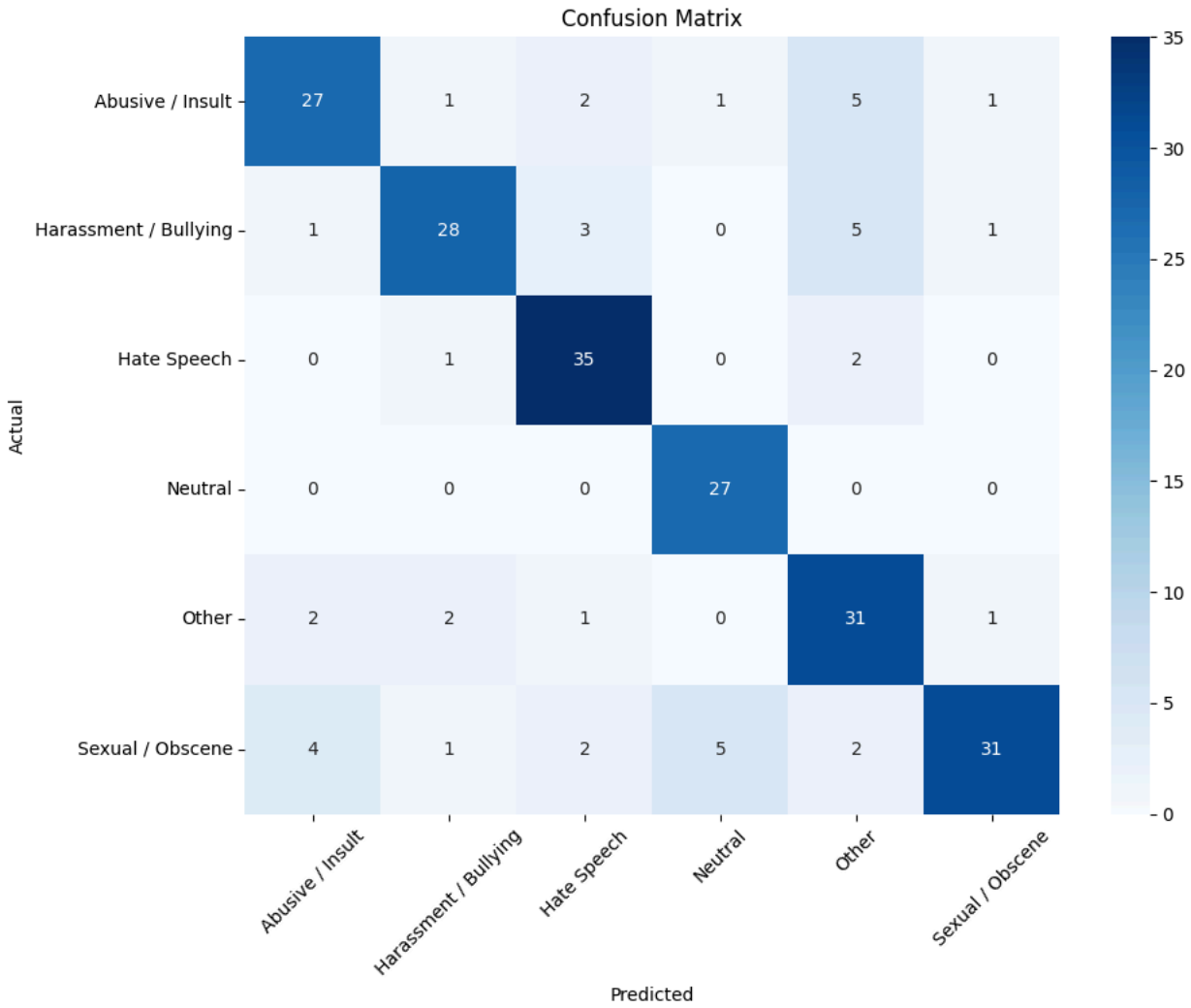



```
🔍 Data Quality Check:
Missing text samples: 0
Missing labels: 0
Removed 0 rows with missing labels

🔄 Preprocessing text data...
✅ Final dataset size: 1108 rows

✅ Data loaded and preprocessed successfully!

📋 Sample data:

Original: You are a bully doola lanja
Processed: you are a bully doola lanja
Category: Harassment / Bullying
----------------------------------------

Original: I hate bloody chettas
Processed: i hate bloody chettas
Category: Other
----------------------------------------

Original: douchebag bastard
Processed: douchebag bastard
Category: Abusive / Insult
----------------------------------------

Original: Today is {day}
Processed: today is {day}
Category: Neutral
----------------------------------------

Original: I hate doxs
Processed: i hate doxs
Category: Harassment / Bullying
----------------------------------------
```

```
# Cell 6: Train the Model
# =======================
```

```
print("🚀 Starting model training...")

try:
    # Train the model
    results = classifier.train_model(X, y, test_size=0.2)

    print(f"\n🎉 Training completed successfully!")
    print(f"Final Accuracy: {results['accuracy']:.4f}")

except Exception as e:
    print(f"❌ Training error: {e}")
```

```
🚀 Starting model training...

============================================================
🚀 TRAINING TOXICITY CLASSIFIER
============================================================
📊 Training set size: 886
📊 Testing set size: 222

🏷️ Number of classes: 6
Classes: ['Abusive / Insult', 'Harassment / Bullying', 'Hate Speech', 'Neutral', 'Other', 'Sexual / Obscene']

🔤 Vectorizing text data...
📐 TF-IDF matrix shape: (886, 397)
📖 Vocabulary size: 397

🤖 Training Logistic Regression model...

✅ Test Accuracy: 0.8063

📊 Classification Report:
============================================================
                      precision    recall  f1-score   support

     Abusive / Insult       0.79      0.73      0.76        37
Harassment / Bullying       0.85      0.74      0.79        38
          Hate Speech       0.81      0.92      0.86        38
              Neutral       0.82      1.00      0.90        27
                Other       0.69      0.84      0.76        37
     Sexual / Obscene       0.91      0.69      0.78        45

             accuracy                           0.81       222
            macro avg       0.81      0.82      0.81       222
         weighted avg       0.82      0.81      0.80       222
```




```
🔄 Cross-Validation Scores:
CV Accuracy: 0.8307 (+/- 0.0554)

🎉 Training completed successfully!
Final Accuracy: 0.8063
```

```
# Cell 7: Test Predictions with Sample Comments
# ==============================================
print("\n" + "="*60)
```

```
print("\n" + "="*60)
print("🧪 TESTING PREDICTIONS")
print("="*60)

# Test comments covering different categories and languages
test_comments = [
    # Neutral comments
    "This is a great video!",
    "I love this content",
    "What a wonderful day",
    "Nenu family istapaduthunnanu",  # Telugu: I love family
    "Today is Friday",

    # Toxic comments from your dataset
    "You are stupid idiot",
    "go to hell bastard",
    "bloody chetta loser",
    "You're such a douchebag",
    "fucking moron",

    # Mixed/Hybrid comments
    "bully doola lanja",
    "stupid erri pooka",

    # Edge cases
    "",  # Empty
    "   ",  # Only spaces
    "https://example.com check this out"  # With URL
]

print(f"Testing {len(test_comments)} sample comments...\n")

for i, comment in enumerate(test_comments, 1):
    try:
        prediction = classifier.predict_comment(comment)

        print(f"🔍 Test {i}:")
        print(f"   Comment: '{comment}'")
        print(f"   Predicted: {prediction['predicted_category']}")
        print(f"   Confidence: {prediction['confidence']:.3f}")

        # Show top 3 probabilities
        top_probs = sorted(prediction['probabilities'].items(),
                           key=lambda x: x[1], reverse=True)[:3]
        print(f"   Top 3 probabilities:")
        for cat, prob in top_probs:
            print(f"     - {cat}: {prob:.3f}")
        print("-" * 50)

    except Exception as e:
        print(f"❌ Error predicting comment {i}: {e}")
```

```
============================================================
🧪 TESTING PREDICTIONS
============================================================
Testing 15 sample comments...

🔍 Test 1:
   Comment: 'This is a great video!'
   Predicted: Neutral
   Confidence: 0.414
   Top 3 probabilities:
     - Neutral: 0.414
     - Sexual / Obscene: 0.156
     - Abusive / Insult: 0.124
--------------------------------------------------
🔍 Test 2:
   Comment: 'I love this content'
   Predicted: Neutral
   Confidence: 0.243
   Top 3 probabilities:
     - Neutral: 0.243
     - Sexual / Obscene: 0.212
     - Abusive / Insult: 0.159
--------------------------------------------------
🔍 Test 3:
   Comment: 'What a wonderful day'
   Predicted: Neutral
   Confidence: 0.485
   Top 3 probabilities:
     - Neutral: 0.485
     - Sexual / Obscene: 0.133
     - Abusive / Insult: 0.109
--------------------------------------------------
🔍 Test 4:
```

```
   Comment: 'Nenu family istapaduthunnanu'
   Predicted: Neutral
   Confidence: 0.527
   Top 3 probabilities:
     - Neutral: 0.527
     - Sexual / Obscene: 0.120
     - Abusive / Insult: 0.100
--------------------------------------------------
🔍 Test 5:
   Comment: 'Today is Friday'
   Predicted: Neutral
   Confidence: 0.467
   Top 3 probabilities:
     - Neutral: 0.467
     - Sexual / Obscene: 0.141
     - Abusive / Insult: 0.112
--------------------------------------------------
🔍 Test 6:
   Comment: 'You are stupid idiot'
   Predicted: Abusive / Insult
   Confidence: 0.562
   Top 3 probabilities:
     - Abusive / Insult: 0.562
     - Neutral: 0.144
```

```python
# Cell 8: Interactive Prediction Testing
# ====================================
print("\n🎮 INTERACTIVE TESTING")
print("="*40)
print("You can now test your own comments!")

def test_custom_comment(comment_text):
    """Helper function to test custom comments"""
    try:
        result = classifier.predict_comment(comment_text)

        print(f"\n📝 Your comment: '{comment_text}'")
        print(f"🎯 Prediction: {result['predicted_category']}")
        print(f"📊 Confidence: {result['confidence']:.3f}")

        print(f"\n📈 All probabilities:")
        for category, prob in sorted(result['probabilities'].items(),
                                     key=lambda x: x[1], reverse=True):
            bar = "█" * int(prob * 20)  # Simple bar chart
            print(f"   {category:20} {prob:.3f} |{bar}")

        return result
    except Exception as e:
        print(f"❌ Error: {e}")
        return None


# Test a few examples
example_tests = [
    "jani mogga cheeku"
]

for test_comment in example_tests:
    test_custom_comment(test_comment)

```

```
🎮 INTERACTIVE TESTING
========================================
You can now test your own comments!

📝 Your comment: 'jani mogga cheeku'
🎯 Prediction: Neutral
📊 Confidence: 0.243

📈 All probabilities:
   Neutral              0.243 |└└└└
   Sexual / Obscene     0.212 |└└└└
   Abusive / Insult     0.159 |└└└
   Other                0.140 |└└
   Hate Speech          0.126 |└└
   Harassment / Bullying 0.120 |└└
```

```python
# Cell 9: Save Model (Optional - for later use)
# ============================================
print("\n💾 SAVING MODEL")
print("="*30)

try:
    # Create a models directory
```